Atharv Mulik Roll no=27 CSE(AI) Div=D Batch =2

Double-click (or enter) to edit

Assignment 6 Use the diabetes data set from UCI and Pima Indians Diabetes data set for performing the following: a. Univariate analysis: Frequency, Mean, Median, Mode, Variance, Standard Deviation, Skewness and Kurtosis b. Bivariate analysis: Linear and logistic regression modeling c. Multiple Regression analysis d. Also compare the results of the above analysis for the two data sets. Dataset link: https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, confusion_matrix
from sklearn.linear_model import LinearRegression, LogisticRegression
```

```
df=pd.read_csv("/content/diabetes.csv")
df
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1
...	...	...	...	...	...	...	...	...	...
763	10	101	76	48	180	32.9	0.171	63	0
764	2	122	70	27	0	36.8	0.340	27	0
765	5	121	72	23	112	26.2	0.245	30	0
766	1	126	60	0	0	30.1	0.349	47	1
767	1	93	70	31	0	30.4	0.315	23	0

768 rows × 9 columns

Next steps: Generate code with df New interactive sheet

```
df.shape
```

```
(768, 9)
```

```
diabetes = load_diabetes()
sklearn_df=pd.DataFrame(diabetes.data,columns=diabetes.feature_names)
sklearn_df['target']=diabetes.target
print("Sklearn Diabetes Shape:", sklearn_df.shape)

sklearn_df.head()
```

```
Sklearn Diabetes Shape: (442, 11)
```

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6	target
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646	151.0
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204	75.0
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930	141.0
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362	206.0
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641	135.0

Next steps: Generate code with sklearn_df New interactive sheet

Univaraite analysis

```
def univariate_analysis(df):
    result = pd.DataFrame({
        "Mean": df.mean(),
        "Median": df.median(),
        "Mode": df.mode().iloc[0],
        "Variance": df.var(),
        "Std Dev": df.std(),
        "Skewness": df.skew(),
        "Kurtosis": df.kurt()
    })
    return result
```

```
print("Univariate Analysis for Diabetes Dataset:")
print(univariate_analysis(df))
print("\nUnivariate Analysis - Sklearn Diabetes Dataset")
print(univariate_analysis(sklearn_df))
```

```
Univariate Analysis for Diabetes Dataset:
                                  Mean    Median    Mode      Variance  \
Pregnancies                   3.845052    3.0000   1.000     11.354056
Glucose                     120.894531  117.0000  99.000   1022.248314
BloodPressure                69.105469   72.0000  70.000    374.647271
SkinThickness                20.536458   23.0000   0.000    254.473245
Insulin                      79.799479   30.5000   0.000  13281.180078
BMI                          31.992578   32.0000  32.000     62.159984
DiabetesPedigreeFunction      0.471876    0.3725   0.254      0.109779
Age                          33.240885   29.0000  22.000    138.303046
Outcome                       0.348958    0.0000   0.000      0.227483

                              Std Dev  Skewness  Kurtosis
Pregnancies                  3.369578  0.901674  0.159220
Glucose                     31.972618  0.173754  0.640780
BloodPressure               19.355807 -1.843608  5.180157
SkinThickness               15.952218  0.109372 -0.520072
Insulin                    115.244002  2.272251  7.214260
BMI                          7.884160 -0.428982  3.290443
DiabetesPedigreeFunction     0.331329  1.919911  5.594954
Age                         11.760232  1.129597  0.643159
Outcome                      0.476951  0.635017 -1.600930

Univariate Analysis - Sklearn Diabetes Dataset
                Mean      Median       Mode     Variance    Std Dev  Skewness  \
age    -1.444295e-18    0.005383   0.016281     0.002268   0.047619 -0.231382
sex     2.543215e-18   -0.044642  -0.044642     0.002268   0.047619  0.127385
bmi    -2.255925e-16   -0.007284  -0.030996     0.002268   0.047619  0.598148
bp     -4.854086e-17   -0.005670  -0.040099     0.002268   0.047619  0.290658
s1     -1.428596e-17   -0.004321  -0.037344     0.002268   0.047619  0.378108
s2      3.898811e-17   -0.003819  -0.001001     0.002268   0.047619  0.436592
s3     -6.028360e-18   -0.006584  -0.013948     0.002268   0.047619  0.799255
s4     -1.788100e-17   -0.002592  -0.039493     0.002268   0.047619  0.735374
s5      9.243486e-17   -0.001947  -0.018114     0.002268   0.047619  0.291754
s6      1.351770e-17   -0.001078   0.003064     0.002268   0.047619  0.207917
target  1.521335e+02  140.500000  72.000000  5943.331348  77.093005  0.440563

        Kurtosis
age    -0.671224
sex    -1.992811
bmi     0.095094
bp     -0.532797
s1      0.232948
s2      0.601381
s3      0.981507
s4      0.444402
s5     -0.134367
s6      0.236917
target -0.883057
```

```
print(df['Outcome'].value_counts())
```

```
Outcome
0    500
1    268
Name: count, dtype: int64
```

Linear regression

```
X = sklearn_df[['bmi']]
y = sklearn_df['target']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

y_pred = lin_reg.predict(X_test)

print("Linear Regression Results (Sklearn Dataset)")
print("Coefficient:", lin_reg.coef_)
print("Intercept:", lin_reg.intercept_)
print("R2 Score:", r2_score(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
```

```
Linear Regression Results (Sklearn Dataset)
Coefficient: [988.41931249]
Intercept: 151.04202449316014
R2 Score: 0.2803417492440603
MSE: 3884.936720961032
```

Logistic Regression

```
X = df[['Glucose']]
y = df['Outcome']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

log_reg = LogisticRegression()
log_reg.fit(X_train, y_train)

y_pred = log_reg.predict(X_test)

print("Logistic Regression Results (Pima Dataset)")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
Logistic Regression Results (Pima Dataset)
Accuracy: 0.7229437229437229
Confusion Matrix:
 [[128  23]
 [ 41  39]]
```

Multi Regression Analysis

```
X = sklearn_df.drop(columns=['target'])
y = sklearn_df['target']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

multi_lin = LinearRegression()
multi_lin.fit(X_train, y_train)

y_pred = multi_lin.predict(X_test)

print("Multiple Linear Regression (Sklearn Dataset)")
print("R2 Score:", r2_score(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
```

```
Multiple Linear Regression (Sklearn Dataset)
R2 Score: 0.4772897164322617
MSE: 2821.750981001311
```

Start coding or generate with AI.